数据与算法

CH07 查找

北京科技大学
计算机与通信工程学院
杜　冰/皇甫伟

本节内容

- 线性查找表
- 散列
- 拓展部分

查找

- **动机：**
  - 某个Node是否访问过？$Visited[i]$
  - 双向BFS的前沿面是否触碰？
  - ……
- **问题**
  - 在某个数据结构中，特定的值是否存在？
  - 在哪里？

查找

- **查找依据：**
  - 等于(=)
  - 大于小于等于(< = >)
- **查找单个的值 VS 查找区间?**
- **基于序关系：线性查找表 查找树**
- **基于（大概率）等值关系：散列**

线性查找表

- 关键字：数据元素内被关注的数据项
- 查找表：待查找数据元素(记录)的集合
- 查找（搜索）：在查找表中寻找特定数据元素，包含与给定关键字匹配的数据项，应用于各类信息检索
- **静态查找**结构：数据集合的组织方式只适合于查找数据元素的**存在与否**，以及数据元素的属性
- **动态查找**结构：除了查找数据元素的存在与属性，还可以高效实现数据元素的**插入与删除**
- 查找方法的评价
  - 时间复杂度：关键字比较的次数
  - 空间复杂度：数据组织所需消耗的额外空间

线性查找表

- **平均查找长度(Average Search Length)**
  - 查找过程中为确定目标数据的位置，关键字比较次数的期望

$$ASL = \sum_{i=0}^{n-1} P_i C_i$$

  - P_i和C_i分别为第i个数据元素被查找的概率和需要比较的次数
  - 若考虑查找失败的情况，需要加以修正
  - 衡量查找性能，**ASL小通常查找性能好**

线性查找表

- 数据元素依次存放在一个线性表中

序 号	学 号	姓 名	班 级
1	2001010150	李定南	通信17
2	2001010151	王炳文	通信14
3	2001010152	张琴	通信11
…	…	…	
219	2001010368	王贵	通信13
220	2001010369	何敏	通信13

- 顺序查找(线性查找)：
  - 从线性查找表的一端开始，依次把每个元素的关键字与查询值进行比较

线性查找表

- 假设表长为n，每个数据元素被查找概率相等，不考虑查找失败：

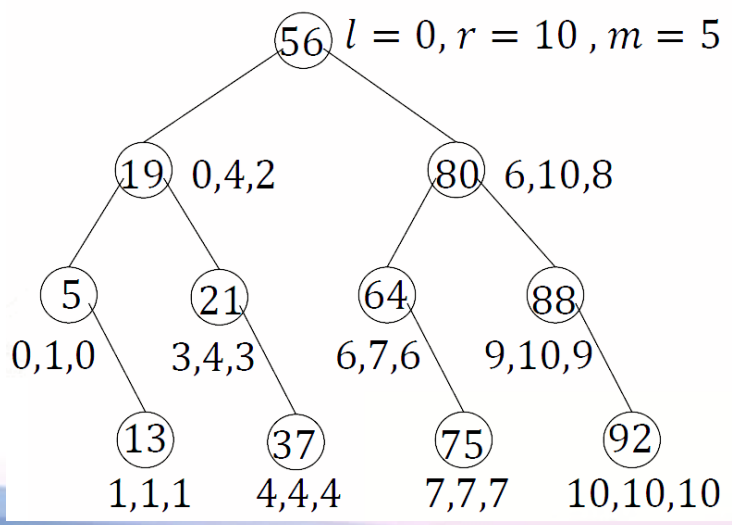
$$ASL = \sum_{i=0}^{n-1} P_i C_i = \sum_{i=0}^{n-1} \frac{1}{n}(i+1) = \frac{n+1}{2}$$

- 对于长度为n的线性查找表，平均意义上顺序查找需要进行**$n/2$次关键字比较**
- 如果查找表无序，**查找失败需要n次比较**
- 如果查找表有序，**查找失败需要$n/2$次比较**

线性查找表 将规模为N的问题降低为规模为N/2的相同问题 → 减治

- **折半查找**
  - 查找表有序，假设关键字从小到大排列
  - 待查关键字和表中间元素关键字比较:
    - 若小则在表前半部折半查找，否在在表后半部折半查找

	0	1	2	3	4	5	6	7	8	9	10
"75"	5	13	19	21	37	56	64	75	80	88	92
						1st	3rd	4th	2nd		



l: 起始位置
r: 终止位置
m: 中间元素位置

折半查找树（BST）

线性查找表

- **有序线性查找表的实现**

```
template <class T>
class orderedSearchList{
  private:
    Node<T>* m_elem;
    int len;
  public:
   searchList(int sz);
   ~searchList()      ;
   void insert(T &x) ;
   int biSearch(T &x);
   int biSearchR(T &x, int l, int r);
};
```

```
template <class T>
class Node{
  public:
    T m_key;
};
```

线性查找表

```
template <class T>
int orderedSearchList<T>::biSearchR(T &x, int l, int r)
{
  int mid = -1;
  if(l<=r){
    mid = (l+r)/2;
    if(m_elem[mid].m_key<x)
       mid = biSearchR(x, mid+1, r);
    if(m_elem[mid].m_key>x)
       mid = biSearchR(x, l, mid-1);
  }
  return mid;
}
```

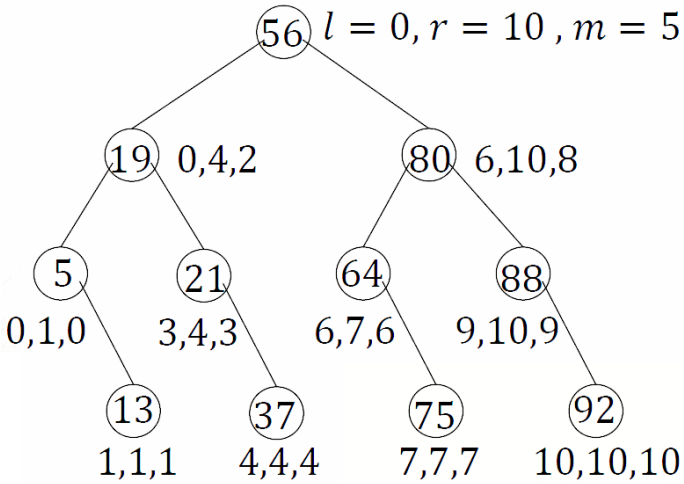
线性查找表

```
template <class T>
int orderedSearchList<T>::biSearchR(T &x, int l, int r)
{
  int mid = -1;
  if(l<=r){
    mid = l+(r-l)/2;
    if(m_elem[mid].m_key<x)
       mid = biSearchR(x, mid+1, r);
    if(m_elem[mid].m_key>x)
       mid = biSearchR(x, l, mid-1);
  }
  return mid;
}
```

S. Meyers 《 Effective C++》
J. Bloch 《Effective Java》
J. Bentley 《Programming Pearls》
… …

线性查找表

- **折半查找树的深度为每次查找需要的比较次数:**



	0	1	2	3	4	5	6	7	8	9	10
	5	13	19	21	37	56	64	75	80	88	92
比较次数	3	4	2	3	4	1	3	4	2	3	4

- 折半查找中一次查找无论成功失败，所需比较次数不超过$\lfloor \log(n) \rfloor + 1$
- $ASL = O(\log(n))$

线性查找表

线性查找表操作时间复杂度比较

	查 找	插 入	删 除
无序顺序表	$O(n)$	$O(1)$ 1	$O(n)$
无序链表	$O(n)$	$O(1)$	$O(n)$
有序顺序表	$O(\log(n))$	$O(n)$	$O(n)$
有序链表	$O(n)$	$O(n)$	$O(n)$

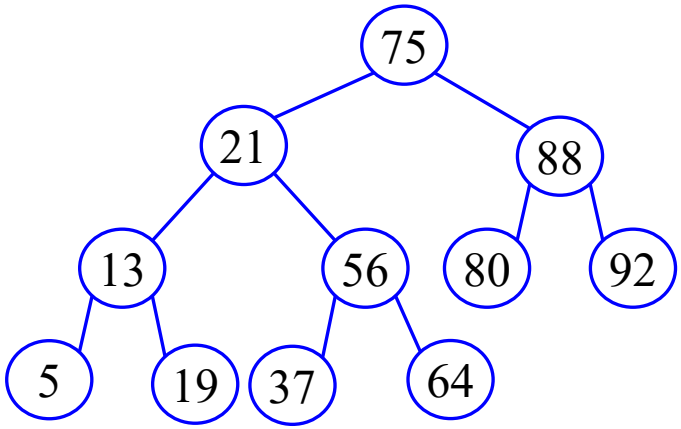
- 有序顺序表查找性能较好，但**插入删除代价高**
- 线性查找表较为适合用于**静态**查找结构

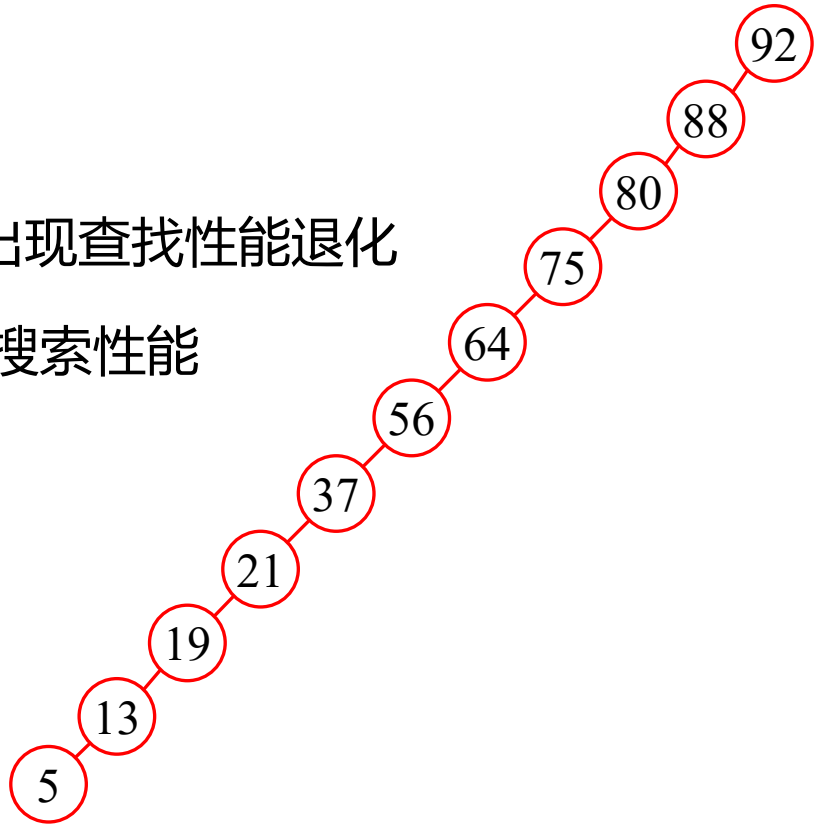
1

- *插入时顺序表放在尾部，链表放在首部即可*
- *删除时所在的位置时任意的，需要找到它并移动后续元素*

搜索树

- **二(多)叉搜索树**
  - 非线性，动态查找结构
  - 树的形态决定操作的复杂度，可能出现查找性能退化
  - 采用AVL/红黑树/B树等，可以保证搜索性能





	5	13	19	21	37	56	64	75	80	88	92
比较次数	4	3	4	2	4	3	4	1	3	2	3
	11	10	9	8	7	6	5	4	3	2	1

线性查找表+搜索树

查找操作时间复杂度比较

	平均情况		最坏情况	
	查找	插入/删除	查找	插入/删除
无序顺序表	$O(n)$	$O(1)/O(n)$	$O(n)$	$O(1)/O(n)$
无序链表	$O(n)$	$O(1)/O(n)$	$O(n)$	$O(1)/O(n)$
有序顺序表	$O(lgn)$	$O(n)/O(n)$	$O(lgn)$	$O(n)/O(n)$
有序链表	$O(n)$	$O(n)/O(n)$	$O(n)$	$O(n)/O(n)$
二叉搜索树	$O(lgn)$	$O(lgn)/O(lgn)$	$O(n)$	$O(n)/O(n)$
AVL/RB树	$\boldsymbol{O(lgn)}$	$\boldsymbol{O(lgn)/O(lgn)}$	$\boldsymbol{O(lgn)}$	$\boldsymbol{O(lgn)/O(lgn)}$

本节内容

- 线性查找表
- 散列
- 拓展部分

散列

- 基于关键字比较的查找结构(线性查找表，二叉搜索树，AVL树等)，时间复杂度$O(lgn)$~$O(n)$
- 若关键字和数据元素的存储位置存在明确的映射关系，则可通过关键字直接访问对应数据元素，**复杂度$O(1)$**

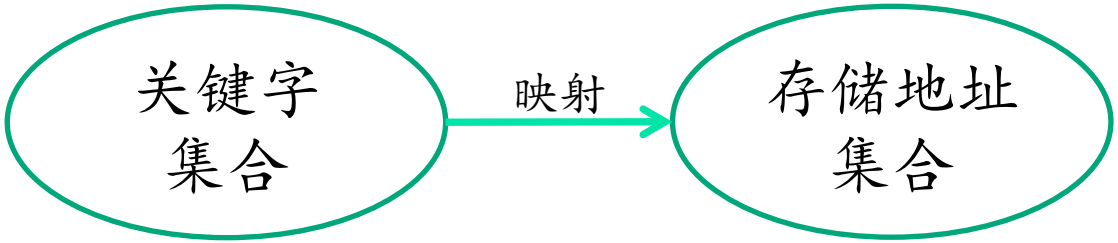
序 号	学 号	姓 名	班 级
1	2001010150	李定南	通信17
2	2001010151	王炳文	通信14
3	2001010152	张琴	通信11
…	…	…	
219	2001010368	王贵	通信13
220	2001010369	何敏	通信13

地址	学 号	姓 名	班 级
150	2001010150	李定南	通信17
151	2001010151	王炳文	通信14
152	2001010152	张琴	通信11
…	…	…	
368	2001010368	王贵	通信13
369	2001010369	何敏	通信13

T. Dean et. al., Fast, Accurate Detection of 100,000 Object Classes on a Single Machine, CVPR 2013 *best paper*

散列

- 散列技术(Hash)：在**数据元素关键字和其存储位置之间建立某种关系**，以得到更好的查找性能



  - 散列函数：通常将关键字映射到有限的连续地址区间
  - 散列表(Hash表)：关键字在地址集合中的”像” 为数据元素在查找表中的存储位置
  - 理想情况：单射，不同关键字映射的地址不同；满射，地址空间完全被占用
  - 实际情况：冲突，不同关键字映射到相同地址，通过冲突调节解决冗余，某些存储地址没有被映射到

散列

- 若Hash表规模远大于数据元素数目，则冲突较易避免(时间复杂度低)，但冗余严重(空间复杂度高)
- 若Hash表规模小，则冗余小但冲突难以避免
- 散列技术力求在空间消耗和查找时间上折中，以达合理的**平衡**
- Hash表的查找、插入的时间复杂度应尽量独立于表的大小而成为**常数**
- 散列技术的关键：**散列函数、冲突处理**

散列

- **散列函数的构造原则**
  - 散列函数应简单，能在较短的时间内计算出结果
  - 散列函数的定义域：所需**全部关键字**
  - 散列函数的值域：如果允许m个地址，则值域：$[0, m-1]$
  - 理想的散列函数应近似为**随机**的，对每一个输入，相应的输出在值域上是**等概率的**，有利于减少冲突

散列

- **乘法散列函数**
  - 对关键字$k \in (0,1)$，乘一个常数：$\mathrm{Hash}(k) = M \times k$

$M = 97$

浮点型关键字	索 引	浮点型关键字	索 引
0.513870656	50	0.277230144	27
0.175725579	17	0.368053228	36
0.308633685	30	0.983458996	95
0.534531713	52	0.535386205	52
0.947630227	92	0.765678883	74
0.171727657	17	0.646473587	63

冲突

冲突

  - 如果关键字$k \in (s,t)$，可使用散列函数：

$$\mathrm{Hash}(k) = M \times (k - s)/(t - s)$$

  - 散列函数的高效性：**关键字在一定范围平均分布**

散列

- **模散列函数**
  - 整型关键字k对常整数M求模得到存储地址：

$$Hash(k) = k \ mod \ M$$

  - $M \leq$ 表长，以免得到的地址超出范围
  - **M应该是质数**，或至少没有小于20的质因子，以减少冲突的出现
  - 结合乘法，关键字乘以常数$a, a \in [0,1]$，然后做模运算：

$$Hash(k) = \lfloor k \times a \rfloor \ mod \ M$$

一个常用的$a = 0.618033 \ldots$（即黄金分割比）

散列

关键字	k%97	k%100	(k*a)%100	关键字	k%97	k%100	(k*a)%100
16838	57	38	6	9084	63	84	14
5758	35	58	58	12060	32	60	53
10113	25	13	50	32225	21	25	16
17515	55	15	24	17543	83	43	42
31051	11	51	90	25089	63	89	5
5627	2	27	77	21183	37	83	91
23010	21	10	20	25137	14	37	35
7419	47	19	85	25566	55	66	0
16212	13	12	19	4978	31	78	76
4086	12	86	**25**	20495	28	95	66
2749	33	49	98	10311	29	11	72
12767	60	67	90	11367	18	67	**25**

散列

- **数字分析法**
  - 设有n个d位的数，每一位可能有r种不同的符号，这些符号在各个位置上出现的频率可能不同。根据表的大小，选取符号中**分布较为均匀的若干位**作为地址
  - 例如学生的学号，前六位分布集中，后三位分布均匀，因此可以直接用来作为存储地址

地址	学 号	姓 名	班 级
150	2001010150	李定南	通信17
151	2001010151	王炳文	通信14
152	2001010152	张屹	通信11
…	…	…	

  - 仅适用于事先**明确知道关键字每位符号的分布**，依赖于特定的关键字集合，不具有普遍性

散列

- **平方取中法**
  - 关键字的标识符内码取平方，按照散列表的大小取中间若干位作为散列地址
  - 平方值的中间几位与关键字每位都相关，因此性能较好，接近“随机化”
  - 缺点是计算比较复杂

标识符	内 码	内码的平方	散列地址
A	01	1	1
A1	0134	20420	042
B	02	4	4
DMAX	04150130	21526443617100	443
AMAX	01150130	135423617100	236

散列

- **折叠法**
  - 关键字自左向右分成位数相等的份，每部分位数与散列地址位数相同，最后一部分位数可以短些
  - 各部分的数据以一定方式叠加，结果作为散列地址
  - **移位法：**各部分末位对齐相加
  - **折叠法：**各部分来回折叠，再对齐相加

$$k = 23938587841 \rightarrow 239,385,878,41$$

移位法： $239 + 385 + 878 + 41 = 1543$

折叠法： $239 + 583 + 878 + 14 = 1714$

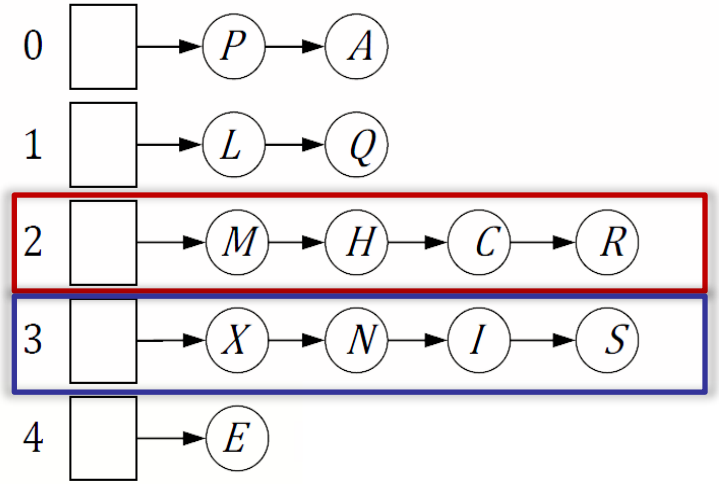
  - 关键字位数多，且每个位上数字分布大致比较均匀时适用

散列

- **散列的冲突处理**
  - 两个关键字散列到同样的地址，即是发生冲突
  - 散列函数的设计可以减少冲突，但往往不能完全避免冲突
  - 冲突处理的方法选取会直接影响性能
  - 冲突处理：**开散列法、闭散列法**

散列

- **开散列法：链地址法**
  - 一种直观的冲突处理办法
  - 对每个散列地址建立一个链表，将散列到同一个地址的不同关键字存储入相应的链表
  - 例：关键字为英文字母，散列函数：



$$Hash(k) = ASCII(k) \bmod 5$$

k	A	S	E	R	C	H	I	N	Q	X	M	P	L
ASCII(k)	65	83	69	82	67	72	73	78	81	88	77	80	76
Hash(k)	0	3	4	2	2	2	3	3	1	3	2	0	1

散列

- **链地址法**
  - 若数据元素的数目为N，链表的个数为M
  - 链表的平均长度：定义负载因子$\alpha = N/M$
  - 散列函数选择合适时，每个链表的长度应接近**α**
  - 相对顺序搜索，**比较次数减少的倍数正比于M**
  - 消耗了M个链表头的额外空间
  - 关于M的选取：
    - M应较小：以避免空的地址链浪费空间
    - M应足够大：以避免链表过长，保证查找效率
    - 一般选取M为数据元素数目的**1/5~1/10**
    - 若存储空间不是关键资源，可以选择较大的M值

散列

- **闭散列法：开放定址法**
  - 若表长M始终大于数据元素的数目N，依靠**表本身的存储空间冗余解决冲突**
  - 探测冲突：检查给定的表位置上是否存在一个与待搜索关键字不同的数据元素
  - **线性探测：**冲突发生时，当前数据元素的插入位置已经被另一个关键字不同的数据元素占据，顺序检查表中下一个位置
  - 利用线性探测的**查找过程**：
    - 如果表位置上有与待查找元素关键字匹配的元素，则命中
    - 如果表位置空则查找失败
    - 如果表位置上数据元素与待查找元素关键字不匹配，继续探测表的后续位置，直到出现上述两种情况之一
    - 若是插入新元素，则放置于使得查找失败的空位置

散列

$$Hash(k) = ASCII(k) \bmod 13$$

插入顺序

k	H	D	J	W	I	R	L	U	X	M	A	V
ASCII(k)	72	68	74	87	73	82	76	85	88	77	65	86
Hash(k)	7	3	9	**9**	8	4	11	**7**	**10**	**12**	0	8

0	1	2	3	4	5	6	7	8	9	10	11	12
X	M	A	D	R	V		H	I	J	W	L	U
X	M	A	D	R			H	I	J	W	L	U
X	M		D	R			H	I	J	W	L	U
X			D	R			H	I	J	W	L	U
			D	R			H	I	J	W	L	U
			D	R			H	I	J	W	L	
			D	R			H	I	J	W		
			D				H	I	J	W		
			D				H		J	W		
			D				H		J			
			D				H					
							H					

散列

- **双重散列**
  - 线性探测，随着表中数据元素的增加，会出现元素聚集的情况，使得探测速度变慢
  - 双重散列不是检查冲突点后面紧邻的位置，而是采用第二个散列函数(再散列函数)，**得到一个固定增量序列用于确定探测序列**
  - 再散列函数的选择：
    - 取值应与表长互素，否则某些探测序列会很短
    - 取值不能为0，否则形成死循环

$$Hash2(k) = \left(ASCII(k) mod\ (M-1)\right) + 1$$

取值范围：$[1, M-1]$；M为素数可以保证互素

散列

- N为数据元素数目，M为表长，对开放定址法定义负载因子 $\alpha = N/M$
- α表示表中位置被元素占用的百分比， $\alpha < 1$
- 开放定址的性能依赖于负载因子α：
  - 当α很小的时候，Hash表为稀疏表，多数查找仅需要很少的探测即可得结果
  - 当α接近1时，Hash表接近满，一次查找探测次数通常很多
  - 一般建议Hash表应低于半满
  - 一般的开放定址比链地址法具有更好的时间性能，但空间复杂度较高

散列

- **散列的删除**
  - 对于链地址法，即是从对应的链表中删除元素
  - 对开放定址散列表，不可以直接删除数据元素

k	H	D	J	W	I	R	L	U	X	M	A	V
ASCII(k)	72	68	74	87	73	82	76	85	88	77	65	86
Hash(k)	7	3	9	9	8	4	11	7	10	12	0	8

0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*
X	*M*	*A*	*D*	*R*	*V*		*H*	*I*	*J*	*W*	*L*	*U*

直接删除*L*

X	*M*	*A*	*D*	*R*	*V*		*H*	*I*	*J*	*W*		*U*

查找*X*

失败！

散列

- **开放定址散列的删除**
  - 元素除了自身信息外，还起到了链接元素的作用
  - 建表过程中，晚于被删除元素插入的元素可能因为发生了冲突而跳过位置
  - 删除元素形成空位，导致某些元素在查找过程中遇到空位错误的终止
  - 可以给被删除关键字加上标志留在表中，而不是真的删除
  - 适当的时机可以对表重新进行散列

散列

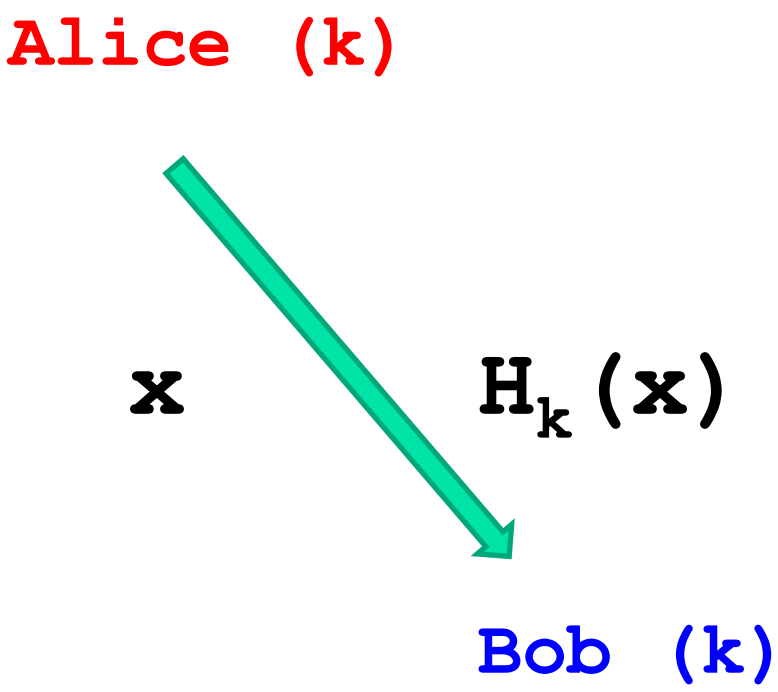
- **散列的性能**
  - 主要取决于**散列函数、冲突处理、表的饱和度(负载因子α)**
  - 若表足够稀疏，线性散列具有最快查找性能
  - 当表趋满时，性能快速下降
  - 双重散列使用内存最为有效，但时间复杂度高
  - 当表趋满时，双重散列性能远优于线性散列
  - 链地址法最容易实现，不会因为元素增加而出现性能快速下降，但平均情况时间性能不如开放定址法

散列

- 散列技术的特点
  - 实现简单，可提供常数时间（$O(1)$）查找性能
  - 散列函数的设计没有固定模式，有些情况下合适的散列函数不易找到
  - **动态性能不好，特别是开放定址散列的删除操作**
  - 最坏情况的查找性能会恶化
  - 忽略数据本身的逻辑联系，**不利于排序等操作**

散列

- **加密算法** MD4, MD5, RIPEMD, SHA-1 …
  - MD5 Message-Digest Algorithm 5
  - 摘要算法 5
  - 用于确保信息传输完整一致
  - 输入不定长度信息，输出固定长度128-bits的算法
  - 经过程序流程，生成四个32位数据，最后联合起来成为一个128-bits散列。基本方式为，求余、取余、调整长度、与链接变量进行循环运算，得出结果



散列

SHA-1 (Secure Hash Algorithm)

在一堆文件查重，可以先查查HASH

- 由NIST(美国国家标准与技术研究院)提出，1993
- 有系列版本：SHA-0/SHA-1/SHA-224 …
- 输入不超过2^{64}bits，输出160bits

People who analyze algorithms have double happiness. First of all they experience the sheer beauty of elegant mathematical patterns that surround elegant computational procedures. Then they receive a practical payoff when their theories make it possible to get other jobs done more quickly and more economically.

ccb3 3530 3fb0 e725 e89d 7c3b 2ed8 0797 d07f 2afc

bbc2 4c1e aafb 0a65 aa00 d7a2 3f5f fb0d 28f3 c47f

people who analyze algorithms have double happiness. First of all they experience the sheer beauty of elegant mathematical patterns that surround elegant computational procedures. Then they receive a practical payoff when their theories make it possible to get other jobs done more quickly and more economically.

本节内容

- 线性查找表
- 散列
- 拓展部分

插值查找

- 仍然考虑一个有序的数组
- 如一本按照音序排列的电话号码簿
- 找“安老师”和“周老师”
- 你会翻到什么地方?

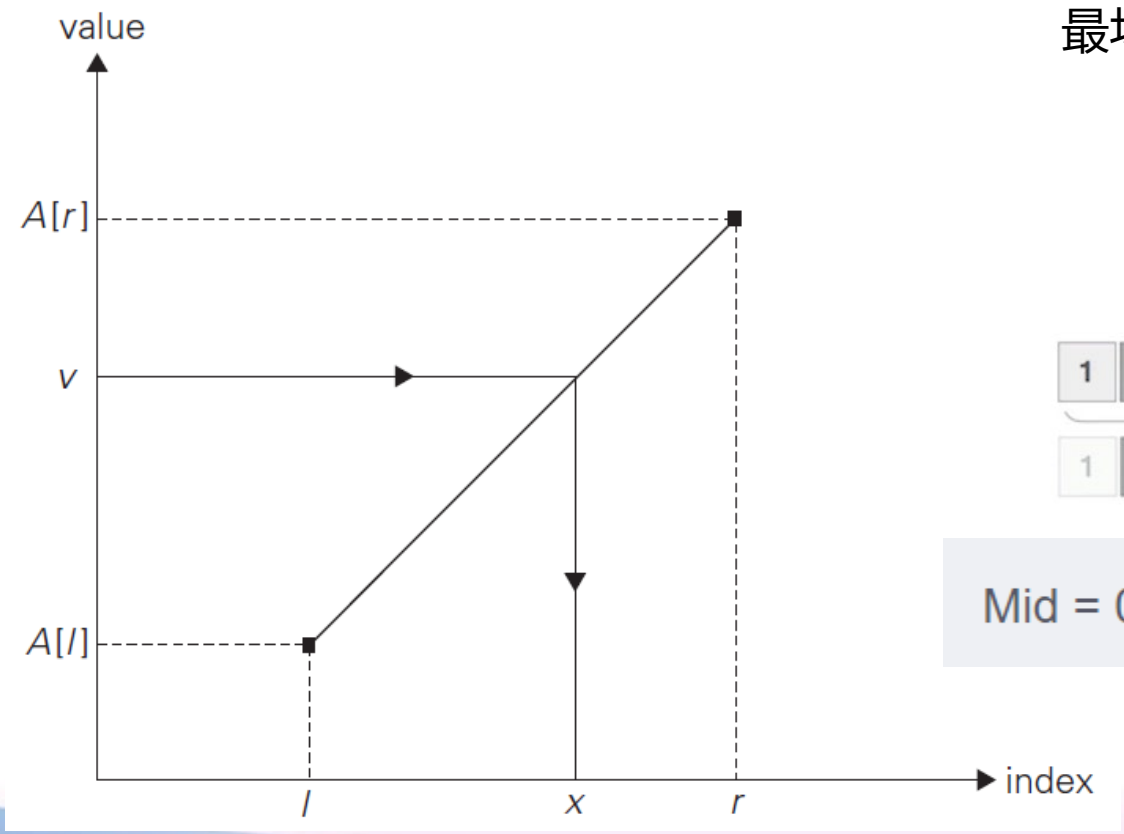
插值查找

插值查找算法只适用于有序序列，作为“改进版”的二分查找算法，当有序序列中的元素呈现均匀分布时，插值查找算法的查找效率要优于二分查找算法；反之，如果有序序列不满足均匀分布的特征，插值查找算法的查找效率不如二分查找算法

- 假设数组内容线性增加
- 效果看假设是否正确（NFL定理）

平均效率 $O(\log\log n)$

最坏情况 $O(n)$

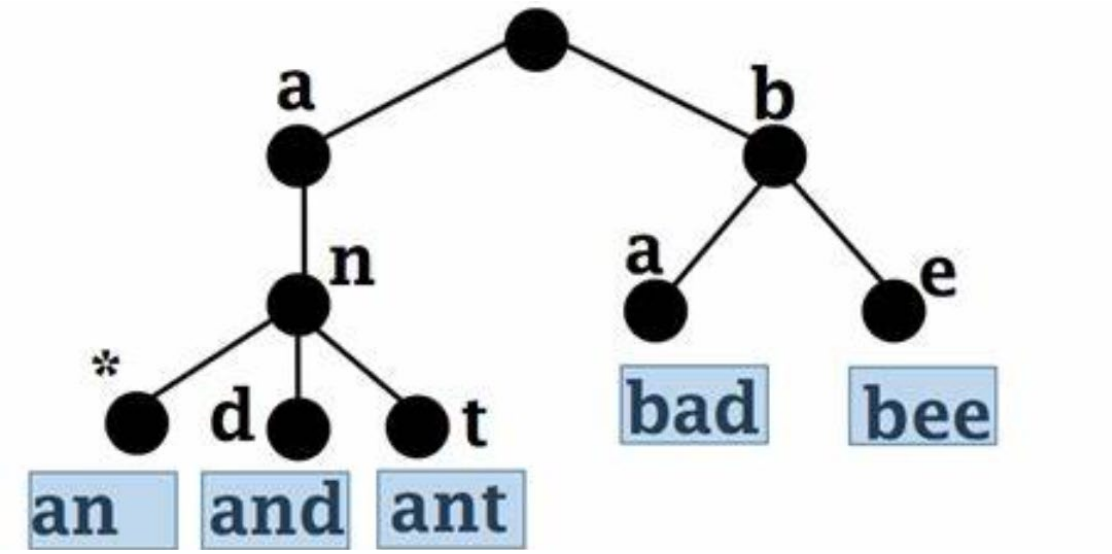


$$x = l + \left\lfloor \frac{(v - A[l])(r - l)}{A[r] - A[l]} \right\rfloor .$$

Mid = 0 + ((9-0)/(10-1)) * (2-1) = 1

多级索引

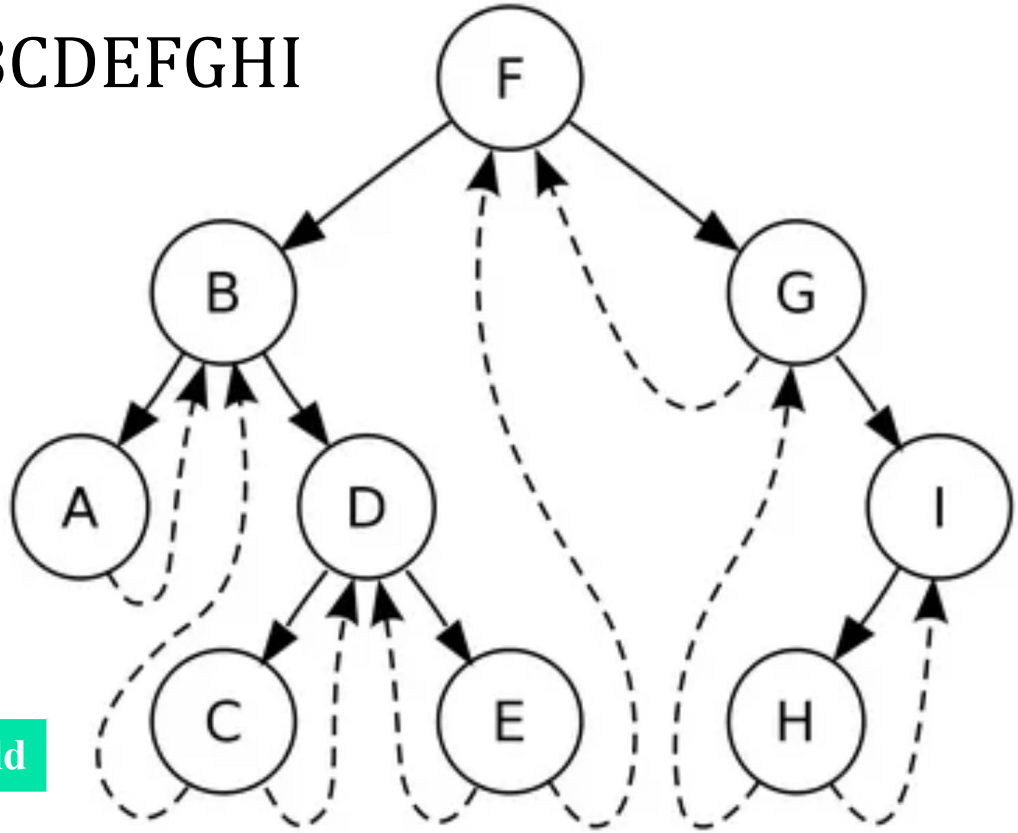
- 字典树（即前缀树、Trie）



  - 利用数据特征，将其分为多个层级，每个层级多个组
  - 例如：学校-院-系-班级……

线索二叉树

ABCDEFGHI



- 更好地支持遍历
- 如中序线索能更好地支持区间查询

LChild	Ltag	data	Rtag	RChild

$$ltag = \begin{cases} 0, & \text{lchild域指示结点的左孩子} \\ 1, & \text{lchild域指示结点的前驱} \end{cases}$$

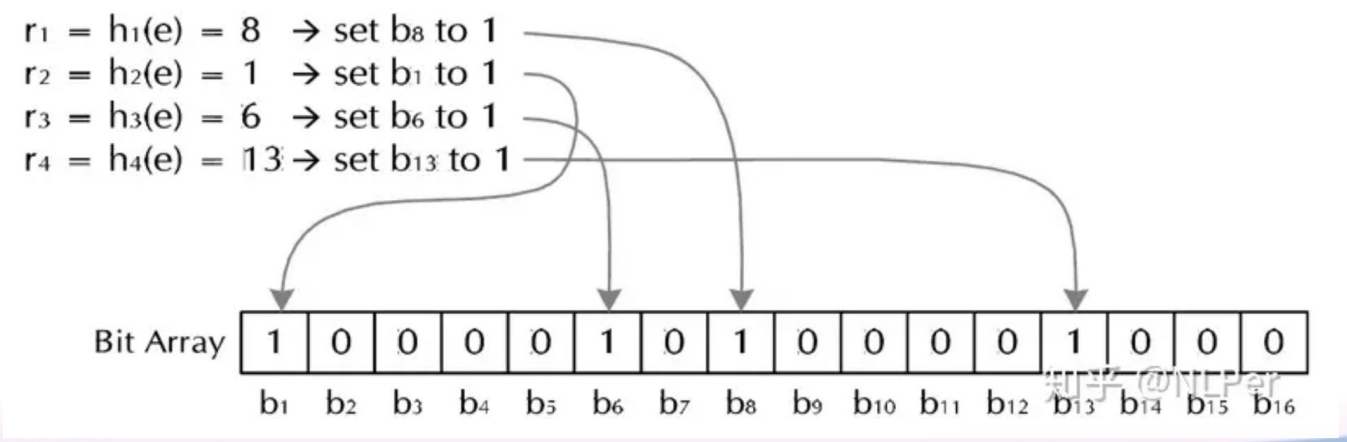
$$rtag = \begin{cases} 0, & \text{rchild域指示结点的右孩子} \\ 1, & \text{rchild域指示结点的后继} \end{cases}$$

- 线索二叉树(Threaded binary tree)：对一棵二叉树中所有结点的空指针域按照某种遍历方式加线索的过程叫作线索化
- 引入线索二叉树的目的是：加快查找结点前驱和后继的速度

Bloom Filter

- 一个大型位数组（二进制数组）+ 多个无偏hash函数
  - 通过k个无偏hash函数计算得到k个hash值依次取模数组长度，得到数组索引，并将对应的数组索引下标位置数据修改为1
  - 键曾经访问过，则所有对应位置为1 （反之概率成立）
  - 任意对应位置为0，则一定没有访问过
  - 允许小概率的出错

布隆过滤器可以用于检索一个元素是否在一个集合中



本章算法思想

- 维护索引结构提升查找效率：有序表、二叉树、散列表等
  - 日常维护精巧的数据结构
  - 磨刀不误砍柴工
- （折半）减治：二分查找法
- 近似： Bloom Filter 布隆滤波器
- 充分利用空间：线索二叉树

数据与算法

谢　谢！

北京科技大学
计算机与通信工程学院